ATMC: what we have done and found

A self-evolving coding assistant on a local model. Results summary for guide review.

1. The project in one minute

- **Goal:** a coding assistant that **learns from its own mistakes without being retrained**.
- **How:** the assistant answers a question, tests run its code, and when it fails, the checker's evidence and a verified fix are saved to a **memory**. Later, similar questions use those notes.
- **ATMC** is our memory design: trust scores, forgetting and merging of notes. The research question is whether those extras beat a simple memory.
- **Model:** Qwen2.5-Coder 7B running **locally on the laptop** (no cloud, no cost). Gemini was used in the earlier experiments.

2. What is built and working

Part	Status
Chat assistant: runs generated code in a sandbox, checks it with tests	Working
Memory (simple and ATMC), automatic answer checking, strategy evolution with rollback	Working
Runs on Gemini or on the local Qwen model	Working
Learning Lab: live side-by-side demo, same questions to an assistant with no memory and one with memory	Working (full live run: 25% vs 36%)
Our own test: 110 hidden-rule questions, 30 unseen held-out questions, two rule-change streams	Built and validated by running every task
Experiment tools with statistics and predictions written before each run	Working; 200 automated tests pass

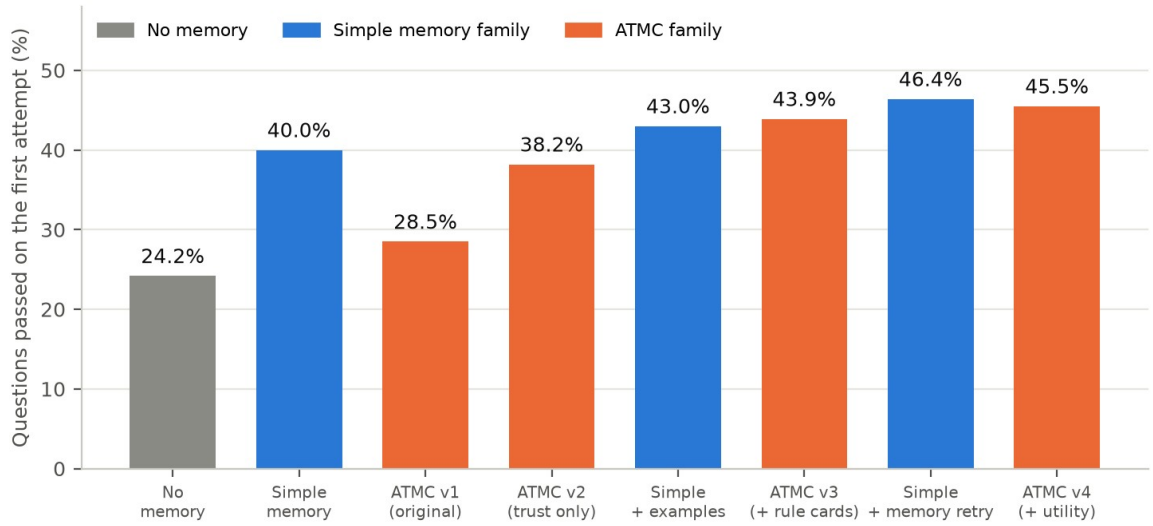
3. How we tested it

- **The test:** 110 coding questions. Each one hides a company rule the question never states (money in paise, dates as DD-MM-YYYY, return None instead of crashing, and so on).
- **Why it is fair:** on 10 ordinary control questions Qwen scored **100%**, so it codes fine; it fails the others only because it does not know the rule. Every task was checked by running a correct solution and a rule-ignoring one.
- **Unseen questions:** questions arrive one at a time, so memory only holds earlier, different questions.
- **Fair comparison:** the same questions in the same order for every system, 3 different orders, paired statistics (95% interval, exact test). Predictions and pass/fail rules were written **before** each run.

4. Results on the main test (3 runs, 330 question comparisons each)

#	Question we asked	Result	What it means
1	Does learning from failures work?	No memory **24.2%**, simple memory **40.0%** (+15.8 points, p < 0.0001)	**Yes.** The core claim holds.
2	Does the original ATMC beat simple memory?	ATMC **28.5%** vs simple 40.0%	No: the extras made it worse.
3	Which ATMC part is responsible?	Trust alone 38.2% (no real change); adding forgetting 30.6% (**-7.6**); adding merging 30.9%	Forgetting deleted useful lessons too early. Trust was neutral.
4	Does trust protect when the checker is wrong 25% / 50% of the time?	+0.6 and +2.4 points, both not significant	Not shown.
5	Add verified worked examples and one retry from the checker's evidence	Simple 43.0% vs ATMC with rule cards 43.9% (+0.9, p = 0.79). About **67%** after the retry	Examples and the retry help every system. ATMC adds nothing measurable.

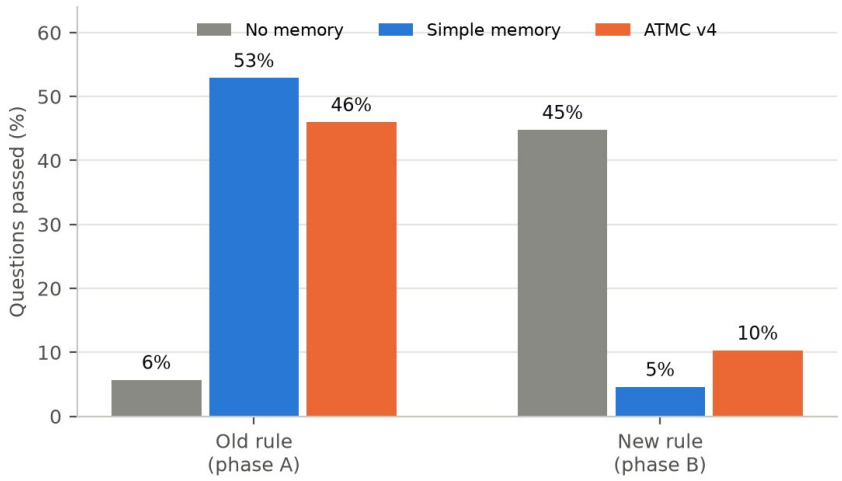
6	Learned utility ranking and a retry that uses memory (v4)	Simple 46.4% vs ATMC 45.5% (-0.9, p = 0.78). About **69%** after the retry	Our pre-set rule (+3 points) was not met.
7	Show only verified notes (abstention)	Simple 46.1% (no change); ATMC **40.0%** (-6.1 vs simple, p = 0.006)	Does not help; it hurt ATMC.



First-attempt pass rate on the 110 hidden-rule questions (average of 3 runs). ATMC improved from 28.5% to 45.5% across versions but never moved past simple memory.

5. When a rule changes (old notes become wrong)

29 questions are asked under an old rule and then again under the **opposite** rule (for example dates change from DD-MM-YYYY to YYYY-MM-DD), so every saved note about them is now wrong. This is the situation trust and forgetting are meant for.



Pass rate before and after the rule flips (average of 3 runs).

- **Simple memory collapses to 5%**, far below having no memory at all (45%): stale notes actively mislead the model.
- **ATMC v4 reaches 10%** (+5.7 points over simple memory, p = 0.125, not significant). It also collapses, because its trust updates are too slow when each note is only shown a few times.
- **In progress:** a re-verification detector. After a failure, every stored solution for the same function is re-run against the current tests; one that now fails is proven stale and its older neighbours are demoted together. Runs are under way. A **second, unseen** rule-change stream is reserved for one final test with nothing changed.

6. What we can and cannot claim

We can claim (backed by data)	We cannot claim yet
A small local model **learns hidden project rules from its own verified failures**: 24% to 40% first attempt, about 69% after one evidence-guided retry	That ATMC beats a simple memory
It works without any cloud model, fully on the laptop, and ordinary questions are not harmed (controls stay 100%)	That trust makes learning safe when the checker is wrong
We know **why** each ATMC part helps or hurts, from controlled tests with predictions fixed in advance	That ATMC handles rule changes (detector still being tested)
A new validated benchmark of hidden conventions, and a live demo of the learning	That the model's weights improved (no training has been done)

Limits: the test is self-written, one 7B model, 3 runs per setting. Because the design was tuned on this test, the 30 unseen questions and the second rule-change stream are reserved for single, final runs.

7. Next steps

- Finish the re-verification detector runs; then one final run on the unseen rule-change stream.
- Run the final design once on the 30 unseen held-out questions.
- Fine-tune the local model on its own verified solutions (the model itself would change), then test on unseen questions with memory switched off.
- Update the paper, novelty analysis, slides and demo with these results.

8. Questions the guide may ask

Question	Short answer
Is the model being trained?	No. Only the memory grows. Fine-tuning is a planned next step.
Is the test rigged?	The hidden rules cannot be guessed, so failures are not coding weakness (controls 100%). It measures learning, not general coding, and we say so. Unseen sets are reserved for final checks.
So is ATMC useless?	Not shown to beat a simple memory on clean data; it improved 17 points across versions. The rule-change test is where it should matter, and it is under test now. The learning itself works either way.
Why a small local model?	It is private, free and unlimited, and it makes real mistakes, so learning is visible. The closest papers mostly use large cloud models.

All numbers come from saved result files in pilot/results and docs/results; experiment notes with the predictions are in docs/experiments.